

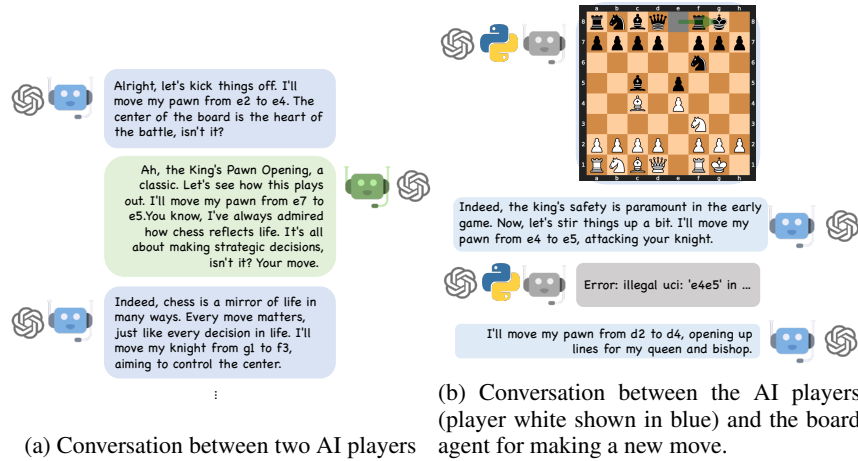


Figure 15: Example conversations during a game involving two AI player agents and a board agent.

To illustrate the effect facilitated by this board agent, we provide a demonstration of conversational chess without a board agent in Figure 16. In this demonstration, instead of employing an additional board agent for grounding, the system utilizes prompting for grounding by including the sentence “*You should make sure both you and the opponent are making legal moves.*” in the system messages directed to both players.

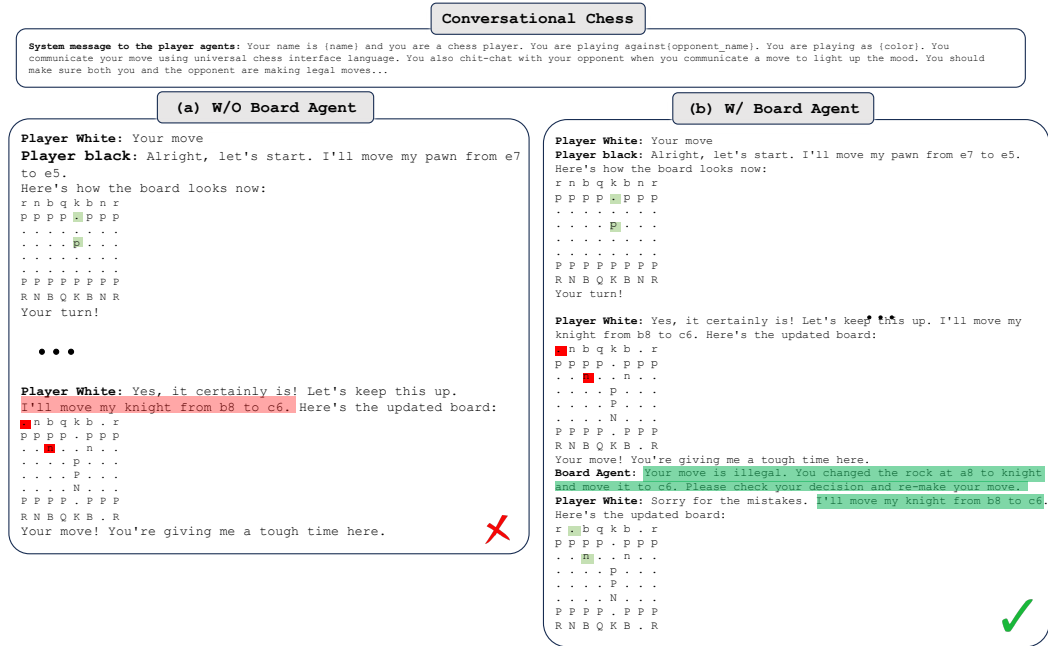


Figure 16: Comparison of two designs—(a) without a board agent, and (b) with a board agent—in Conversational Chess.

A7: Online Decision Making for Browser interactions

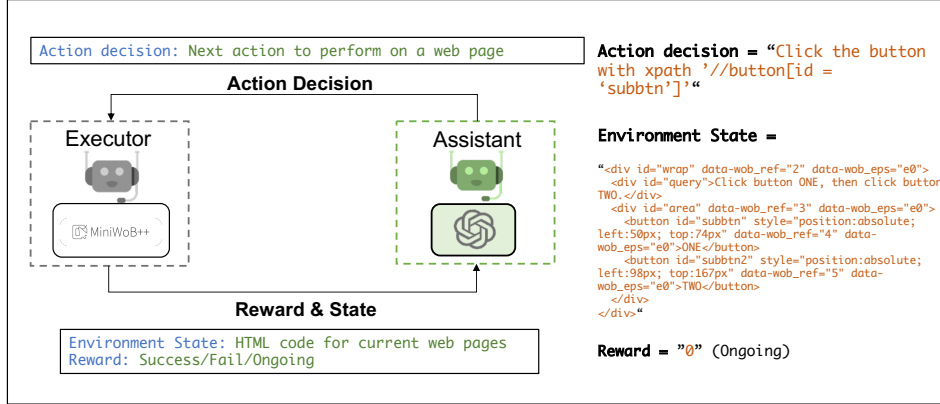


Figure 17: We use AutoGen to build MiniWobChat, which solves tasks in the MiniWoB++ benchmark. MiniWobChat consists of two agents: an assistant agent and an executor agent. The assistant agent suggests actions to manipulate the browser while the executor executes the suggested actions and returns rewards/feedback. The assistant agent records the feedback and continues until the feedback indicates task success or failure.

In practice, many applications require the presence of agents capable of interacting with environments and making decisions in an online context, such as in game playing (Mnih et al., 2013; Vinyals et al., 2017), web interactions (Liu et al., 2018; Shi et al., 2017), and robot manipulations (Shen et al., 2021). With the multi-agent conversational framework in AutoGen, it becomes easy to decompose the automatic agent-environment interactions and the development of a decision-making agent by constructing an *executor* agent responsible for handling the interaction with the environment, thereby delegating the decision-making part to other agents. Such a decomposition allows developers to reuse the decision-making agent for new tasks with minimal effort rather than building a specialized decision-making agent for every new environment.

Workflow. We demonstrate how to use AutoGen to build a working system for handling such scenarios with the MiniWoB++ benchmark (Shi et al., 2017). MiniWoB++ comprises browser interaction tasks that involve utilizing mouse and keyboard actions to interact with browsers. The ultimate objective of each task is to complete the tasks described concisely in natural language, such as “expand the web section below and click the submit button.” Solving these tasks typically requires a sequence of web manipulation actions rather than a single action, and making action decisions at each time step requires access to the web status (in the form of HTML code) online. For the example above, clicking the submit button requires checking the web status after expanding the web section. We designed a straightforward two-agent system named MiniWobChat using AutoGen, as shown in Figure 17. The assistant agent is an instance of the built-in `AssistantAgent` and is responsible for making action decisions for the given task. The second agent, the executor agent, is a customized `UserProxyAgent`, which is responsible for interacting with the benchmark by executing the actions suggested by the `AssistantAgent` and returning feedback.

To assess the performance of the developed working system, we compare it with RCI (Kim et al., 2023), a recent solution for the MiniWoB++ benchmark that employs a set of self-critiquing prompts and has achieved state-of-the-art performance. In our evaluation, we use all available tasks in the official RCI code, with varying degrees of difficulty, to conduct a comprehensive analysis against MiniWobChat. Figure 18 illustrates that MiniWobChat achieves competitive performance in this evaluation⁸. Specifically, among the 49 available tasks, MiniWobChat achieves a success rate of 52.8%, which is only 3.6% lower than RCI, a method specifically designed for the MiniWoB++ benchmark. It is worth noting that in most tasks, the difference between the two methods is mirrored as shown in Figure 18. If we consider 0.1 as a success rate tolerance for each task, i.e., two methods that differ within 0.1 are considered to have the same performance, both methods outperform the

⁸We report the results of RCI by running its official code with default settings.

other on the same number of tasks. For illustration purposes, we provide a case analysis in Table 7 on four typical tasks.

Additionally, we also explored the feasibility of using Auto-GPT for handling the same tasks. Auto-GPT faces challenges in handling tasks that involve complex rules due to its limited extensibility. It provides an interface for setting task goals using natural language. However, when dealing with the MiniWob++ benchmark, accurately instructing Auto-GPT to follow the instructions for using MiniWob++ proves challenging. There is no clear path to extend it in the manner of the two-agent chat facilitated by AutoGen.

Takeaways: For this application, AutoGen stood out as a more user-friendly option, offering modularity and programmability: It streamlined the process with autonomous conversations between the assistant and executor, and provided readily available solutions for agent-environment interactions. The built-in AssistantAgent was directly reusable and exhibited strong performance without customization. Moreover, the decoupling of the execution and assistant agent ensures that modifications to one component do not adversely impact the other. This convenience simplifies maintenance and future updates.

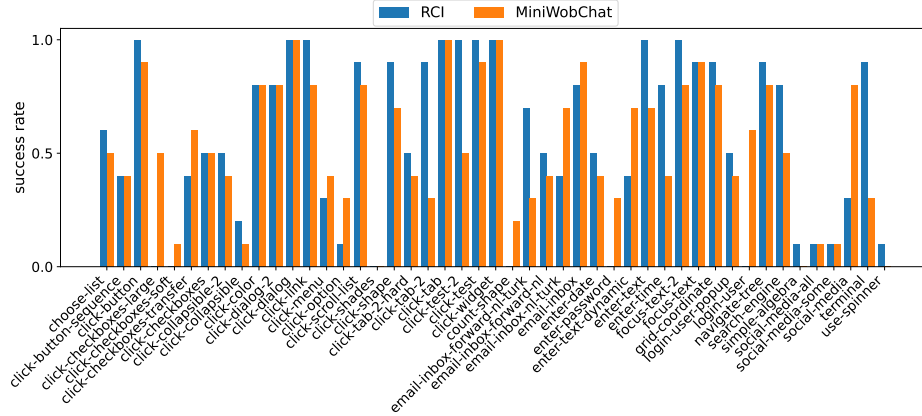


Figure 18: Comparisons between RCI (state-of-the-art prior work) and MiniWobChat on the MiniWob++ benchmark are elucidated herein. We utilize all available tasks in the official RCI code, each with varying degrees of difficulty, to conduct comprehensive comparisons. For each task, the success rate across ten different instances is reported. The results reveal that MiniWobChat attains a performance comparable to that of RCI. When a success rate tolerance of 0.1 is considered for each task, both methods outperform each other on an equal number of tasks.

Table 7: Cases analysis on four typical tasks from MiniWob++.

	Correctness	Main failure reason
click-dialog	AutoGen: 10/10 RCI: 10/10	N/A. N/A.
click-checkboxes-large	AutoGen: 5/10 RCI: 0/10	AssistantAgent provides actions with infeasible characters. RCI performs actions that are out of its plan.
count-shape	AutoGen: 2/10 RCI: 0/10	AssistantAgent provide actions with redundant content that can not convert to actions in the benchmark. RCI provides a wrong plan in most cases.
use-spinner	AutoGen: 0/10 RCI: 1/10	AssistantAgent return actions out of its plan. RCI provides a wrong plan in most cases.

E Example outputs from applications

In this section, we include example outputs from the following applications and systems:

- Application A1: autonomous solving process of one problem with: ChatGPT + Plugin (Table 8), AutoGen (Table 9), LangChain ReAct (Table 10), AutoGPT (Table 11), Multi-Agent Debate (Table 12), and ChatGPT + Code Interpreter (Table 14).
- Application A4: an OptiGuide problem with AutoGen (Table 15) and ChatGPT + Code Interpreter (Table 13).
- Application A1: preliminary evaluation on several alternative multi-agent systems: BabyAGI (Table 16), CAMEL (Table 17), and MetaGPT (Table 18).

Table 8: Application A1, Scenario 1: Autonomous math problem-solving. ChatGPT+ Plugin (Wolfram Alpha) on the first qualitative example. The response from Wolfram contains the correct answer, but ChatGPT selected the wrong one.

	Action	ChatGPT + Plugin
	Input	Completely simplify and rationalize the denominator: $\frac{\sqrt{160}}{\sqrt{252}} \times \frac{\sqrt{245}}{\sqrt{108}}$
		REQUEST TO WOLFRAM: <pre> 1 { 2 "input": "simplify (sqrt(160)/sqrt(252))* 3 (sqrt(245)/sqrt(108))" 4 }</pre> RESPONSE FROM WOLFRAM <pre> 1 Query: 2 "simplify (sqrt(160)/sqrt(252))*(sqrt(245)/sqrt(108))" 3 4 Input interpretation: 5 simplify sqrt(160)/sqrt(252)x sqrt(245)/sqrt(108) 6 7 Results: 8 (5 sqrt(42))/27 9 10 (5 sqrt(14/3))/9 11 12 Decimal approximation: 13 1.2001371663718259686974013770533327143898526494552... 14 15 Wolfram Alpha website result for "simplify 16 (sqrt(160)/sqrt(252))*(sqrt(245)/sqrt(108))": 17 [link]</pre>
		The completely simplified and rationalized form of the given expression is: $\frac{5\sqrt{14/3}}{9}$ In decimal form, this is approximately 1.200137166371825968697401377053332714...

Table 9: Application A1, Scenario 1: Autonomous math problem-solving. AutoGen on the first qualitative example. AutoGen solves the problem correctly.

	Action	AutoGen
	Init chat	Completely simplify and rationalize the denominator: $\frac{\sqrt{160}}{\sqrt{252}} \times \frac{\sqrt{245}}{\sqrt{108}}$
		We can simplify and rationalize this denominator using Python’s symbolic mathematics library ‘sympy’. Here is the Python code to resolve this: <pre> 1 from sympy import sqrt, Rational 2 3 # Define the entire fraction as a rational number 4 fraction = (sqrt(160)/sqrt(252))*(sqrt(245)/sqrt(108)) 5 6 # Simplify the fraction 7 simplified = fraction.simplify() 8 9 # Print the simplified fraction 10 print(simplified) </pre> This code performs the requested simplification by constructing the fraction using the ‘Rational()’ function from the ‘sympy’ module, which automatically simplifies fractions when possible. The ‘sqrt()’ function from the ‘sympy’ module enables square root calculations. Finally, the ‘simplify()’ function simplifies expressions when possible.
	Auto Reply	exitcode: 0 (execution succeeded) Code output: 5*sqrt(42)/27
		TERMINATE

Table 10: Application A1, Scenario 1: Autonomous math problem-solving. LangChain ReAct on the first qualitative example. The generated code does not match the devised plan. The code returns a decimal number.

	Action	LangChain ReAct
	Input	Completely simplify and rationalize the denominator: $\frac{\sqrt{160}}{\sqrt{252}} \times \frac{\sqrt{245}}{\sqrt{108}}$
		Entering new AgentExecutor chain... To simplify this expression, I will first simplify each square root individually, then multiply the fractions as normal. The result should be a rational number. Action: Python.REPL Action Input: <pre> 1 import math 2 3 # Calculate square roots 4 sqrt_160 = math.sqrt(160) 5 sqrt_252 = math.sqrt(252) 6 sqrt_245 = math.sqrt(245) 7 sqrt_108 = math.sqrt(108) 8 9 # Multiply fractions 10 numerator = sqrt_160 * sqrt_245 11 denominator = sqrt_252 * sqrt_108 12 result = numerator / denominator 13 14 print(result) </pre> Observation: 1.200137166371826 Thought: I now know the final answer Final Answer: The simplified and rationalized form of the expression is approximately 1.200137166371826. Finished chain.